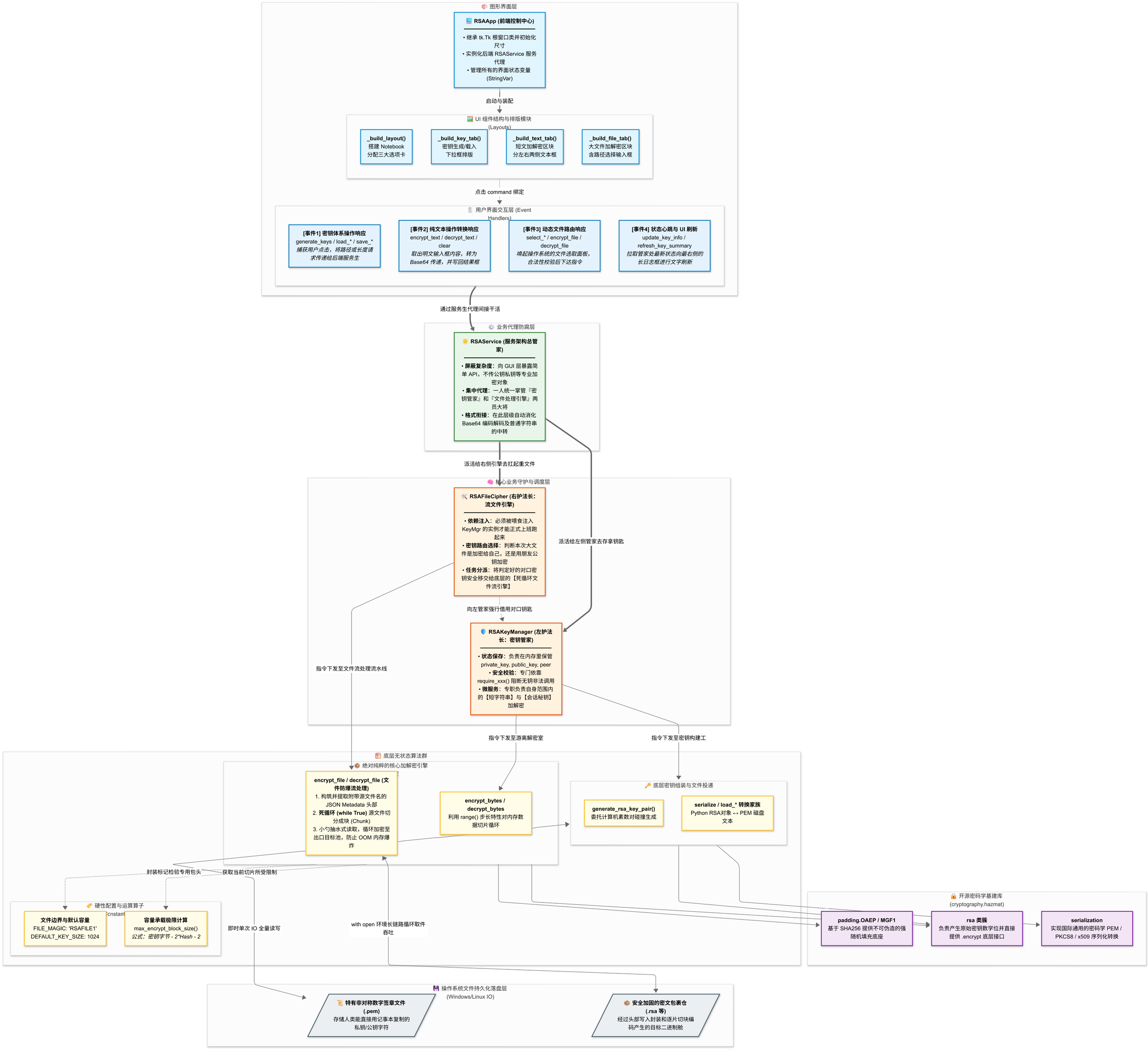
图形界面层
RSAApp (前端控制中心)
• 继承 tk.Tk 根窗口类并初始化尺寸
• 实例化后端 RSAService 服务代理
• 管理所有的界面状态变量 (StringVar)
启动与装配
UI 组件结构与排版模块 (Layouts)
_build_layout() 搭建 Notebook 分配三大选项卡
_build_key_tab() 密钥生成/载入 下拉框排版
_build_text_tab() 短文加解密区块 分左右两侧文本框
_build_file_tab() 大文件加解密区块 含路径选择输入框
点击 command 绑定
用户界面交互层 (Event Handlers)
[事件1] 密钥体系操作响应 generate_keys / load_* / save_* 捕获用户点击，将路径或长度请求传递给后端服务生
[事件2] 纯文本操作转换响应 encrypt_text / decrypt_text / clear 取出明文输入框内容，转为 Base64 传递，并写回结果框
[事件3] 动态文件路由响应 select_* / encrypt_file / decrypt_file 唤起操作系统的文件选取面板，合法性校验后下达指令
[事件4] 状态心跳与 UI 刷新 update_key_info / refresh_key_summary 拉取管家处最新状态向最右侧的长日志框进行文字刷新
通过服务生代理间接干活
业务代理防腐层
RSAService (服务架构总管家)
• 屏蔽复杂度：向 GUI 层暴露简单 API，不传公钥私钥等专业加密对象
• 集中代理：一人统一掌管『密钥管家』和『文件处理引擎』两员大将
• 格式衔接：在此层级自动消化 Base64 编码解码及普通字符串的中转
派活给右侧引擎去扛起重文件
派活给左侧管家去存拿钥匙
核心业务守护与调度层
RSAFileCipher (右护法长：流文件引擎)
• 依赖注入：必须被喂食注入 KeyMgr 的实例才能正式上班跑起来
• 密钥路由选择：判断本次大文件是加密给自己，还是用朋友公钥加密
• 任务分派：将判定好的对口密钥安全移交给底层的【死循环文件流引擎】
向左管家强行借用对口钥匙
RSAKeyManager (左护法长：密钥管家)
• 状态保存：负责在内存里保管 private_key, public_key, peer
• 安全校验：专门依靠 require_xxx() 阻断无钥非法调用
• 微服务：专职负责自身范围内的【短字符串】与【会话秘钥】加解密
指令下发至文件流处理流水线
指令下发至游离解密室
指令下发至密钥构建工
底层无状态算法群
绝对纯粹的核心加解密引擎
encrypt_file / decrypt_file (文件防爆流处理)
1. 构筑并提取附带源文件名的 JSON Metadata 头部
2. 死循环 (while True) 源文件切分成块 (Chunk)
3. 小勺抽水式读取，循环加密至出口目标池，防止 OOM 内存爆炸
encrypt_bytes / decrypt_bytes
利用 range() 步长特性对内存数据切片循环
底层密钥组装与文件投递
generate_rsa_key_pair()
委托计算机素数对碰撞生成
serialize / load_* 转换家族
Python RSA对象 ↔ PEM 磁盘文本
封装标记检验专用包头
获取当前切片所受限制
硬性配置与运算算子
文件边界与默认容量
FILE_MAGIC: 'RSAFILE1'
DEFAULT_KEY_SIZE: 1024
容量承载极限计算
max_encrypt_block_size()
公式：密钥字节 - 2*Hash - 2
即时单次 IO 全量读写
with open 环境长链路循环取件吞吐
开源密码学基建库 (cryptography.hazmat)
padding.OAEP / MGF1
基于 SHA256 提供不可伪造的强随机填充底座
rsa 类簇
负责产生原始密钥数学位并直接提供 .encrypt 底层接口
serialization
实现国际通用的密码学 PEM / PKCS8 / x509 序列化转换
操作系统文件持久化落盘层 (Windows/Linux IO)
特有非对称数字签章文件 (.pem)
存储人类能直接用记事本复制的私钥/公钥字符
安全加固的密文包裹仓 (.rsa 等)
经过头部写入封装和逐片切块编码产生的目标二进制舱